**Computer Architecture Lab 2**

Jonas Van Nieuwenhuyze, Gil Nimmegeers

# 1 Introduction

Maximum value detection is a fundamental operation in computational tasks. In this report, we explore three approaches for maximum value detection: a nested for loop on the CPU, a GPU kernel employing atomic operations, and a GPU kernel using reduction. We assess the correctness and performance of these approaches for varying input array sizes.

# 2 Implementation

The CPU implementation follows a traditional and obvious approach.

In the GPU implementation using atomic operations, parallelism is harnessed to expedite the maximum value detection process. Each thread within the GPU kernel is assigned a portion of the input array, and atomicMax() operations are employed to ensure the thread-safe updating of the maximum value.

The GPU kernel utilizing reduction implements a parallel reduction technique to efficiently identify the maximum value. Threads within each block collaboratively work to find the maximum value within their designated section of the input array. Through successive reduction steps across multiple blocks, the global maximum value is determined. Both GPU kernels are timed using CUDA events (cudaEventRecord, cudaEventSynchronize, cudaEventElapsedTime).

# 3 Evaluation

The program functions correctly for array sizes up to 2048, but beyond this threshold, correctness relies on chance. Notably, operational stability is compromised around an array size of 15,000. Synchronization across blocks is essential for larger sizes, as the __synchronize() function only synchronizes threads within a single block, not across all blocks. In terms of performance, the CPU implementation exhibits linear execution time, whereas GPU implementations demonstrate nearly constant execution time. This discrepancy underscores the significant speed advantage of GPU over CPU computation. (see figure 1)
Surprisingly, the GPU Atomic implementation outperforms the GPU Reduction approach, contrary to initial expectations. However, this anomaly can be attributed to the basic nature of the reduction technique employed. Further optimization of the reduction process holds promise for unlocking additional performance gains.
(https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf).

Figure 1: Timing CPU, GPU Atomic, GPU Reduction in ms

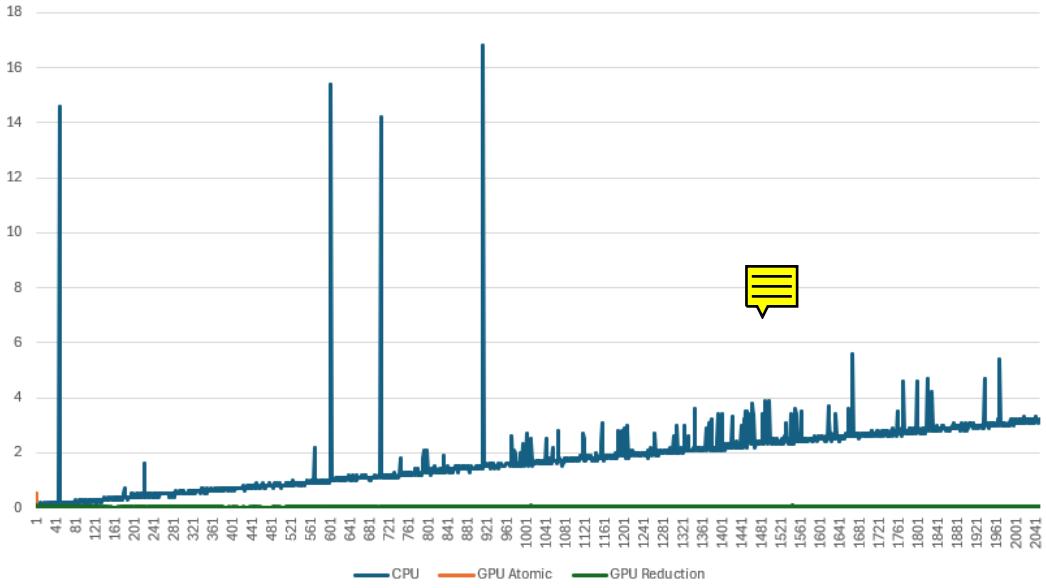



Figure 2: Timing GPU Atomic, GPU Reduction in ms

